

NATIONAL UNIVERSITY OF MODERN LANGUAGES

Faculty of Engineering and Computing
Department of Software Engineering



Operating System

Assignment 02

Submitted To : **Mr Iftikhar Ali**

Submitted By : Ijlal Ahmad
FL-21565

Semester : 4th

Date of Submission : **08 December 2024**

Deadlock

Deadlock refers to a situation where a set of processes are unable to proceed because each process is waiting for a resource that is held by another process in the set. This leads to a state where none of the processes can continue execution, thus halting progress in the system.

The four necessary conditions for deadlock

Mutual Exclusion:

A resource can only be assigned to one process at a time, or if it is shared, it can only be used by one process at a time. If a process is holding a resource, other processes must wait for that resource to become available.

Hold and Wait:

A process that is holding at least one resource is allowed to wait for additional resources that are being held by other processes. This condition allows processes to hold some resources while waiting for others.

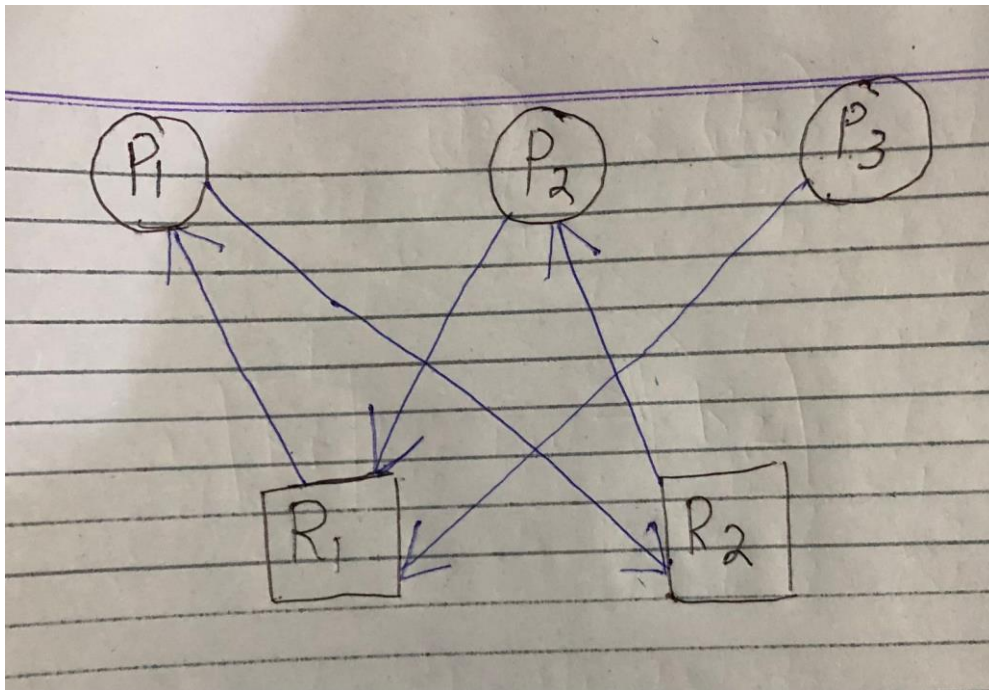
No Preemption:

Resources cannot be forcibly removed from processes holding them. Once a process acquires a resource, it must release it voluntarily. Preemption (forcing a process to release a resource) is not allowed.

Circular Wait:

A set of processes exist such that each process in the set is waiting for a resource that is held by the next process in the set. This forms a circular chain of waiting processes, making it impossible for any process to proceed.

b.



Analysis of the RAG:

Resource Allocation:

P1 holds R1, so there is an edge from R1 to P1.

P2 holds R2, so there is an edge from R2 to P2.

Requests:

P1 is requesting R2: There's an edge from P1 to R2.

P2 is requesting R1: There's an edge from P2 to R1.

P3 is requesting R1: There's an edge from P3 to R1.

Circular Wait:

There is no circular wait in this graph.

P1 is requesting R2, which P2 holds.

P2 is requesting R1, which P1 holds.

P3 is requesting R1, but there's no circular waiting for it as it is directly requesting R1.

No deadlock is present in this scenario, as there is no circular waiting between the processes.

C.

	Process	Allocation			Max			Available			Need		
		A	B	C	A	B	C	A	B	C	A	B	C
4x	P ₀	0	1	0	7	5	3	3	3	2	7	4	3
1x	P ₁	2	0	0	3	2	2	5	3	2	1	2	2
5x	P ₂	3	0	2	9	0	2	7	4	3	6	0	0
2x	P ₃	2	1	1	2	2	2	7	4	5	0	1	1
3x	P ₄	0	0	2	4	3	3	7	5	5	4	3	1
								10	4	5			

Need = Max - allocation

Safe Sequence: P₁, P₃, P₄, P₀, P₅

The system is in a safe state because a safe sequence exists where all processes can execute without causing a deadlock.